Kolumnowe bazy danych cz. II

Zaawansowana analityka i dowód wydajności

Imię i nazwisko: Karolina Węgrzyn, Patrycja Markiewicz

Grupa: 1

Cel ćwiczenia

Po tym laboratorium będziesz potrafił:

1. zbudować lejek konwersji i zinterpretować odpływ między jego etapami,
2. użyć funkcji okna do rankingu i analizy trendu przychodów w czasie,
3. podzielić użytkowników na segmenty metodą RFM i wyciągnąć z tego wnioski biznesowe,
4. zmierzyć i wyjaśnić różnicę wydajności między ClickHouse a PostgreSQL i powiedzieć, skąd ona wynika.

Laboratorium ma celowo prostą strukturę: **zadania 1–3 to analityka w ClickHouse**, **zadanie 4 to dowód** eksperyment porównawczy oparty na zapytaniach, które właśnie napisałeś.

Zanim zaczniesz - przeczytaj to uważnie

Której bazy używamy i kiedy

Zadanie	Baza danych
0. Gotowość	obie — sprawdzasz środowisko
1. Lejek konwersji	ClickHouse
2. Funkcje okna	Jedna baza do wyboru
3. Segmentacja RFM	Do wyboru ale ClickHouse lepiej
4. Benchmark i wnioski	obie — porównanie obowiązkowe

W tym laboratorium ClickHouse jest bazą wiodącą. PostgreSQL pojawia się w zadaniu 4 jako punkt odniesienia — po to, żeby różnicę wydajności zmierzyć, nie zakładać.

Jak korzystać ze ściągi

Do zajęć dołączona jest ściąga sql_postgresql_clickhouse_sciaga.pdf. Zawiera składnię i podpowiedzi do przykładów dla każdego zadania. Korzystaj z niej jak z dokumentacji - żeby sprawdzić składnię funkcji, nie żeby skopiować rozwiązanie. Numer sekcji ściągi podany jest przy każdym zadaniu.

Oceniane są interpretacja i komentarz. Sam fakt uruchomienia zapytania nie jest rozwiązaniem.

Sprawozdanie

Oddaj jako PDF albo Markdown. Dla każdego zadania dołącz:

- kod zapytania,
- wynik - tabela lub zrzut ekranu,
- komentarz - pełnymi zdaniami, nie listą słów.

Termin oddania: do końca dnia poprzedzającego kolejne zajęcia.

Punktacja: razem 10 pkt.

Założenie startowe

Środowisko Docker działa, tabela events jest dostępna w obu bazach. Jeżeli tabela nie jest widoczna w bieżącym kontekście, użyj pełnej nazwy:

- public.events w PostgreSQL,
- ds_lab.events w ClickHouse.

0. Gotowość do zajęć — warunek konieczny (0 pkt)

Pokaż, że środowisko działa: kontenery postgres i clickhouse mają status Up, tabela events jest widoczna w obu bazach, połączenie z klienta SQL działa.

Brak gotowości środowiska uniemożliwia wykonanie dalszych zadań.

1. Lejek konwersji — 2 pkt

Dlaczego to robimy

W e-commerce każda sesja przechodzi przez etapy: wyświetlenie → koszyk → zakup. Na każdym etapie część sesji odpada. **Lejek konwersji** mierzy, ile sesji przechodzi przez każdy etap i gdzie odpływ jest największy - to fundamentalny wskaźnik analityki e-commerce. Przy okazji zobaczysz, jak ClickHouse upraszcza agregacje warunkowe dzięki countIf zamiast klasycznego CASE WHEN. (zobacz odpowiednie sekcje w ściądze)

Wykonaj w ClickHouse

Dla każdej sesji ustal, czy wystąpiło w niej zdarzenie view, cart i purchase. Na tej podstawie policz dla całego zbioru:

- liczbę sesji z view,
- liczbę sesji z cart,
- liczbę sesji z purchase,
- wskaźnik cart / view — jaka część sesji z view dotarła do cart,
- wskaźnik purchase / cart — jaka część sesji z cart zakończyła się zakupem,
- wskaźnik purchase / view — ogólna konwersja od pierwszego kontaktu do zakupu.

Następnie powtórz analizę w przekroju device.

Zapytanie startowe

```
WITH session_flags AS (
    SELECT
        session_id,
        countIf(event_type = 'view') > 0 AS has_view,
        countIf(event_type = 'add_to_cart') > 0 AS has_cart,
        countIf(event_type = 'purchase') > 0 AS has_purchase
    FROM events
    GROUP BY session_id
)
-- Krok 2: agregacja do poziomu całego zbioru
SELECT
    countIf(has_view) AS view_sessions,
    countIf(has_cart) AS cart_sessions,
    countIf(has_purchase) AS purchase_sessions,
    countIf(has_cart) / nullIf(countIf(has_view), 0) AS cart_per_view,
    countIf(has_purchase) / nullIf(countIf(has_cart), 0) AS purchase_per_cart,
    countIf(has_purchase) / nullIf(countIf(has_view), 0) AS purchase_per_view
FROM session_flags;
```

	view_sessions	cart_sessions	purchase_sessions	cart_per_view	purchase_per_cart	purchase_per_view
1	196262	105629	44601	0.538204033383946	0.42224199793617284	0.227252346353344

```
WITH session_flags AS (
    SELECT
        session_id,
        any(device) AS device,
        countIf(event_type = 'view') > 0 AS has_view,
        countIf(event_type = 'add_to_cart') > 0 AS has_cart,
        countIf(event_type = 'purchase') > 0 AS has_purchase
    FROM events
    GROUP BY session_id
```

```sql
)

SELECT
    device,
    countIf(has_view) AS view_sessions,
    countIf(has_cart) AS cart_sessions,
    countIf(has_purchase) AS purchase_sessions,
    countIf(has_cart) / nullIf(countIf(has_view), 0) AS cart_per_view,
    countIf(has_purchase) / nullIf(countIf(has_cart), 0) AS purchase_per_cart,
    countIf(has_purchase) / nullIf(countIf(has_view), 0) AS purchase_per_view
FROM session_flags
GROUP BY device;
```

Output | Result 8

	device	view_sessions	cart_sessions	purchase_sessions	cart_per_view	purchase_per_cart	purchase_per_view
1	desktop	65504	35198	15063	0.5373412310698583	0.42795045173021196	0.22995542256961407
2	mobile	65634	35170	14842	0.5358503214797209	0.42200739266420245	0.22613279702593167
3	tablet	65124	35261	14696	0.5414440144954241	0.4167777431156235	0.2256618143848658

Zbuduj analogicznie wersję z GROUP BY device.

W komentarzu napisz

- Na którym etapie lejka odpływ jest największy i co to oznacza dla biznesu — gdzie sklep traci klientów?

Największy odpływ w lejku występuje między pierwszym a drugim etapem. Prawie połowe klientów (46%) sklep traci zaraz po pierwszej interakcji - po oglądnięciu produktu klient nie dodaje go do koszyka.

- Czy lejek różni się między urządzeniami? Jeśli tak, co może być tego przyczyną?

Lejek między urządzeniami jest bardzo podobny. Współczynniki przejścia dla poszczególnych etapów różnią się tylko o kilka procent między sobą. Może to wynikać z drobnych różnic w wygodzie korzystania z interfejsu, wielkości ekranu lub sposobie przeglądania strony, ale ogólnie zachowanie użytkowników jest podobne.

- countIf zastępuje klasyczny wzorzec MAX(CASE WHEN ... THEN 1 ELSE 0 END) ze standardowego SQL. W 2–3 zdaniach wyjaśnij, na czym polega różnica w podejściu i dlaczego wersja ClickHouse jest krótsza.

countIf w ClickHouse pozwala bezpośrednio policzyć liczbę wierszy spełniających dany warunek w funkcji agregującej. W standardowym SQL stosuje się MAX(CASE WHEN warunek THEN 1 ELSE 0 END), aby najpierw zamienić warunek na wartości 0 lub 1, a dopiero potem je zagregować. W ClickHouse podejście jest krótsze i czytelniejsze, bo warunek można bezpośrednio przekazać do countIf.

2. Funkcje okna: rankingi i trend przychodów 2 pkt

Dlaczego to robimy

Zwykłe GROUP BY zwija dane - dostajesz jeden wiersz na grupę. **Funkcje okna** liczą wartości w kontekście innych wierszy bez zwijania wyników: ranking krajów, suma narastająca, zmiana dzień do dnia bez zagnieżdżonych podzapytań.

Wybierz jedną bazę i napisz w niej obie części

Zaznacz w sprawozdaniu, którą bazę wybrałeś.

Część A - Ranking krajów

Policz łączny przychód z zakupów dla każdego kraju i nadaj krajom ranking według przychodu. Użyj RANK() albo DENSE_RANK().

Wynik powinien zawierać kolumny: country, revenue, rank_no.

Wskazówka: funkcję okna możesz zastosować bezpośrednio w SELECT obok agregacji - nie musisz pisać podzapytania ani CTE.

```sql
select
    country,
    sum(price * quantity) as revenue,
    rank() over (order by sum(price * quantity) desc) as rank_no
from events
where event_type = 'purchase'
group by country
order by rank_no;
```

	country	revenue	rank_no
1	FR	1805210.3199999991	1
2	IT	1779036.9599999995	2
3	PL	1774036.4399999997	3
4	NL	1753906.4699999993	4
5	SE	1749563.0100000019	5
6	GB	1748815.37	6
7	NO	1733380.0399999998	7
8	ES	1707084.7800000005	8
9	US	1703552.5700000003	9
10	DE	1694584.4000000008	10

Część B - Narastający przychód i zmiana dzień do dnia

Policz dzienny przychód ze zdarzeń purchase. Następnie w tym samym zapytaniu oblicz:

- sumę narastającą - cumulative_revenue,
- przychód z poprzedniego dnia - prev_day_revenue,
- zmianę procentową względem poprzedniego dnia - pct_change.

Wynik powinien zawierać kolumny: day, daily_revenue, cumulative_revenue, prev_day_revenue, pct_change.

Wskazówki składniowe

Element	PostgreSQL	ClickHouse
Data	DATE(event_time)	toDate(event_time)
Suma narastająca	SUM(x) OVER (ORDER BY day)	sum(x) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
Poprzednia wartość	LAG(x, 1) OVER (ORDER BY day)	lagInFrame(x, 1) OVER (ORDER BY day ROWS ...)

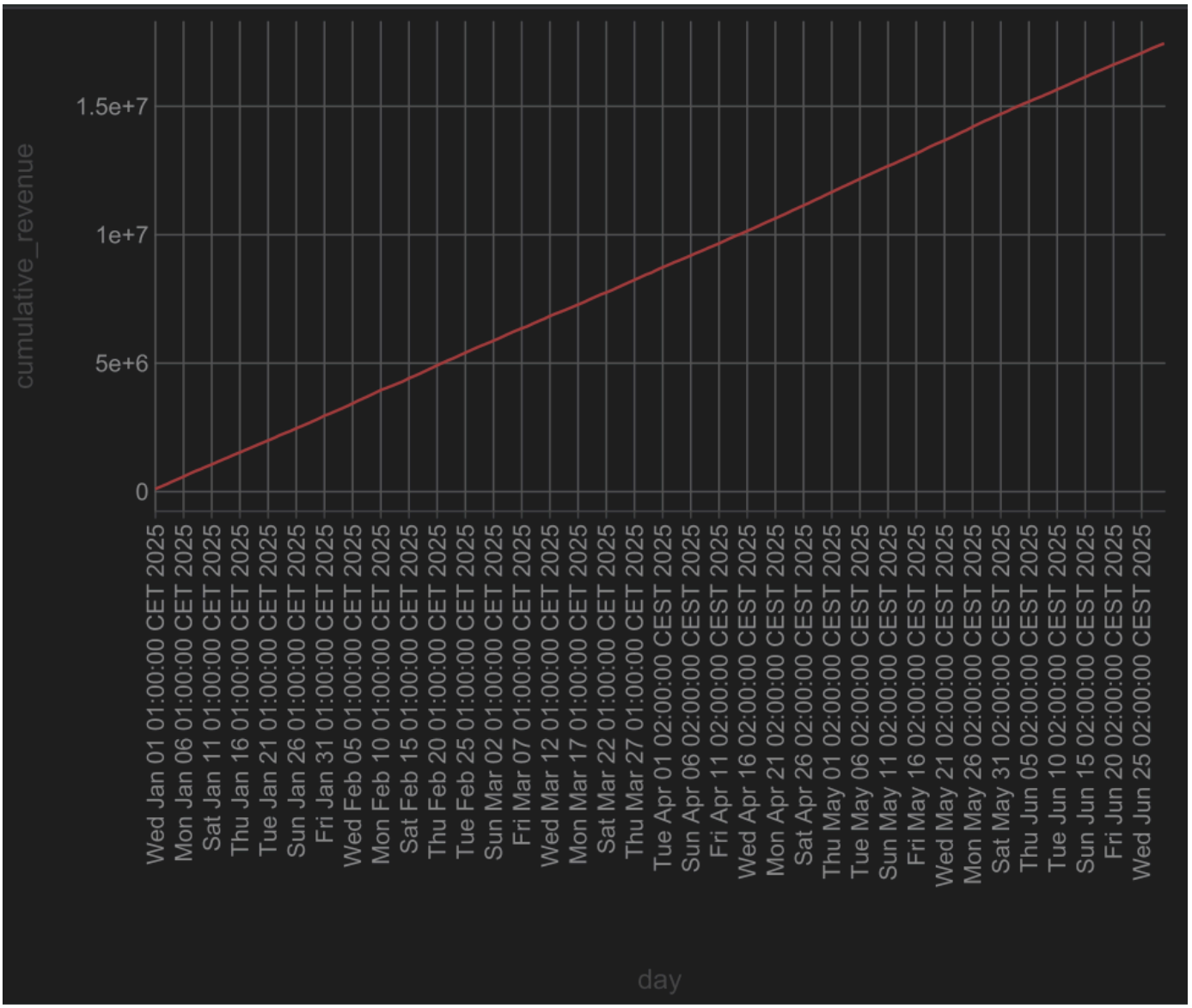
Wskazówka: zbuduj zapytanie w dwóch krokach - najpierw CTE z dziennym przychodem (GROUP BY day), a dopiero do niego dołącz funkcje okna.

```
with daily as (
    select
        toDate(event_time) as day,
        sum(quantity * price) as daily_revenue
    from events
    where event_type = 'purchase'
    group by day
),
daily_with_lag as (
    select
        day,
        round(daily_revenue, 2) as daily_revenue,
        round(sum(daily_revenue) over (order by day rows between unbounded preceding and current row), 2)
as cumulative_revenue,
        lagInFrame(daily_revenue, 1) over (order by day rows between unbounded preceding and current row)
as prev_day_revenue
    from daily
)
select
    day,
    daily_revenue,
    cumulative_revenue,
    prev_day_revenue,
    round(((daily_revenue / prev_day_revenue) - 1) * 100, 2) as pct_change
from daily_with_lag
order by day;
```

W komentarzu napisz

- Czy suma narastająca rośnie równomiernie czy skokowo?

	day	daily_revenue	cumulative_revenue	prev_day_revenue	pct_change
1	2025-01-01	104891.28	104891.28	0	Infinity
2	2025-01-02	93236.65	198127.93	104891.28	-11.11
3	2025-01-03	90552.35	288680.28	93236.65	-2.88
4	2025-01-04	105215.48	393895.76	90552.35	16.19
5	2025-01-05	93124.21	487019.97	105215.48	-11.49
6	2025-01-06	98140.54	585160.51	93124.21	5.39
7	2025-01-07	104057.11	689217.62	98140.54	6.03
8	2025-01-08	99637.44	788855.06	104057.11	-4.25
9	2025-01-09	83591.25	872446.31	99637.44	-16.1
10	2025-01-10	97070.77	969517.08	83591.25	16.13
11	2025-01-11	87207.76	1056724.84	97070.77	-10.16
12	2025-01-12	100450.14	1157174.98	87207.76	15.18
13	2025-01-13	92504.93	1249679.91	100450.14	-7.91
14	2025-01-14	90904.93	1340584.84	92504.93	-1.73



Suma narastająca rośnie równomiernie. Wykres ma kształt niemal idealnie prostej linii, bez widocznych skoków. Dzienny przychód oscyluje wokół stałego poziomu ~90-105 tys, co przekłada się na równomierne tempo narastania.

- Czy widać konkretny dzień z wyraźną zmianą - ile wyniósł wzrost lub spadek w procentach?

Żeby znaleźć skrajne wartości, posortowałam wyniki po `pct_change`.

Największy spadek:

```
order by pct_change
```

	day	daily_revenue	cumulative_revenue	prev_day_revenue	pct_change
1	2025-02-01	81550.44	3039480.12	114908.34	-29.03

Największy spadek: 2025-02-01, pct_change = -29.03%

Największy wzrost:

```
order by pct_change desc
```

	day	daily_revenue	cumulative_revenue	prev_day_revenue	pct_change
1	2025-01-01	104891.28	104891.28	0	Infinity
2	2025-03-31	113003.08	8635224.03	80070.12	41.13

Największy wzrost: 2025-03-31, pct_change = +41.13%

- Co było dla Ciebie nowe w funkcjach okna i co sprawiło największą trudność?

Nowe było zagnieżdżanie funkcji okna na wyniku agregacji, intuicyjnie chcialam pisać wszystko w jednym SELECT, a dopiero wskazówka uświadomiła mi, że agregacja musi być gotowa zanim zastosujemy okno. Trudniejsza okazała się składnia ClickHouse np dla `lagInFrame` z jawną ramką ROWS, której PostgreSQL nie wymaga.

3. Segmentacja użytkowników - metoda RFM - 2 pkt

Dlaczego to robimy

RFM to prosta i skuteczna metoda segmentacji klientów stosowana w marketingu od dekad:

- **R**ecency - jak dawno użytkownik ostatnio kupił?
- **F**requency - ile razy kupił?
- **M**onetary - ile łącznie wydał?

Na tej podstawie dzielisz klientów na segmenty: najlepszych nagradzasz, uśpionych reaktywujesz, nowych zachęcasz do kolejnego zakupu.

Wybierz jedną bazę i wykonaj zadanie w niej

Zaznacz w sprawozdaniu, którą bazę wybrałeś.

Krok 1 — oblicz R, F, M dla każdego użytkownika

```sql
WITH ref AS (
    SELECT max(event_time) AS ref_time
    FROM events
),
     purchases AS (
         SELECT
             user_id,
             count(*) AS frequency,
             sum(price * quantity) AS monetary,
             max(event_time) AS last_purchase_time
         FROM events
         WHERE event_type = 'purchase'
         GROUP BY user_id
     )

SELECT
    p.user_id,
    dateDiff('day', p.last_purchase_time, ref.ref_time) AS recency,
    p.frequency,
    p.monetary
FROM purchases p
         CROSS JOIN ref
ORDER BY monetary DESC;
```

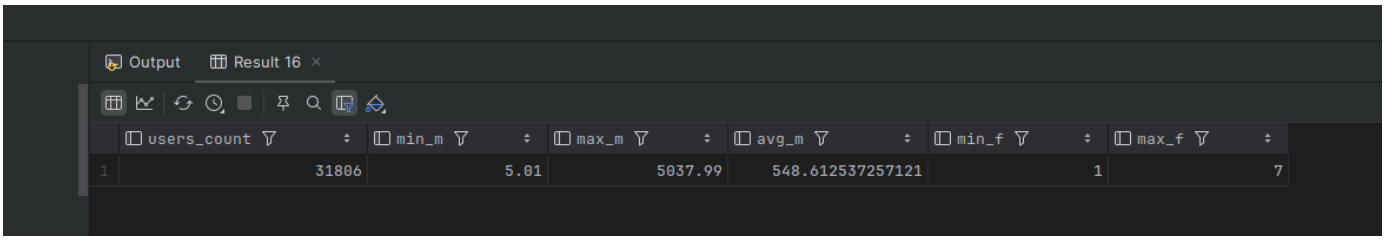
Output - zadanie 3

	user_id	recency	frequency	monetary
1	21194	85	4	5037.99
2	11919	49	3	4859.679999999999
3	18860	53	3	4639.92
4	21978	15	3	4604.17
5	39652	49	3	4584.65
6	35735	69	5	4527.26
7	32592	29	4	4526.099999999999
8	48742	17	5	4460.98
9	4935	76	4	4360.120000000001
10	9978	20	3	4301.870000000001
11	33799	5	2	4204.8
12	29614	2	7	4192.2
13	36059	20	4	4160.82
14	7178	16	4	4119.37
15	37756	19	5	4078.3500000000004
16	23725	25	3	4006.11
17	7009	9	3	4001.56
18	3112	90	4	3980.3599999999997
19	47934	3	4	3915.71
20	48299	157	3	3866.2000000000003
21	37388	46	3	3864.27
22	9063	44	3	3837.99
23	32906	65	2	3830.1899999999996
24	17971	26	2	3795.8500000000004
25	7483	118	5	3775.5800000000004
26	8579	94	5	3758.6699999999996

Zanim przejdziesz dalej — sprawdź zakres wartości, żeby świadomie dobrać progi:

```sql
WITH purchases AS (
    SELECT
        user_id,
        count() AS frequency,
        sum(price * quantity) AS monetary
    FROM events
    WHERE event_type = 'purchase'
    GROUP BY user_id
)

SELECT
    count() AS users_count,
    min(monetary) AS min_m,
    max(monetary) AS max_m,
    avg(monetary) AS avg_m,
    min(frequency) AS min_f,
    max(frequency) AS max_f
FROM purchases;
```



Output | Result 16

	users_count	min_m	max_m	avg_m	min_f	max_f
1	31806	5.01	5037.99	548.612537257121	1	7

Krok 2 — podziel użytkowników na segmenty

Na podstawie wyników z kroku 1 zbuduj trzy segmenty według kolumny monetary. Progi dobierz samodzielnie na podstawie rozkładu danych z powyższego kroku.

```sql
WITH purchases AS (
    SELECT
        user_id,
        count() AS frequency,
        sum(price * quantity) AS monetary
    FROM events
    WHERE event_type = 'purchase'
    GROUP BY user_id
)

SELECT
    user_id,
    frequency,
    monetary,
    CASE
        WHEN monetary >= 1500 THEN 'premium'
        WHEN monetary >= 300 THEN 'standard'
        ELSE 'okazjonalny'
        END AS segment
FROM purchases;
```

Dla każdego segmentu policz:

- liczbę użytkowników,
- łączny przychód segmentu,
- udział procentowy w całkowitym przychodzie wszystkich użytkowników.

```sql
WITH purchases AS (
    SELECT
        user_id,
        sum(price * quantity) AS monetary
    FROM events
    WHERE event_type = 'purchase'
    GROUP BY user_id
),

     segmented AS (
         SELECT
             user_id,
             monetary,
             CASE
                 WHEN monetary >= 1500 THEN 'premium'
                 WHEN monetary >= 300 THEN 'standard'
                 ELSE 'okazjonalny'
                 END AS segment
         FROM purchases
     ),
```

```sql
    segment_stats AS (
        SELECT
            segment,
            count() AS users_count,
            sum(monetary) AS segment_revenue
        FROM segmented
        GROUP BY segment
    ),

    total AS (
        SELECT
            count() AS total_users,
            sum(monetary) AS total_revenue
        FROM purchases
    )

SELECT
    s.segment,
    s.users_count,
    round(s.users_count / t.total_users * 100, 2) AS users_share_pct,
    s.segment_revenue,
    round(s.segment_revenue / t.total_revenue * 100, 2) AS revenue_share_pct
FROM segment_stats s
        CROSS JOIN total t
ORDER BY s.segment_revenue DESC;
```

Output | Result 24

	segment	users_count	users_share_pct	segment_revenue	revenue_share_pct
1	standard	18420	57.91	11350283.969999943	65.05
2	premium	2089	6.57	4300185.1499999985	24.64
3	okazjonalny	11297	35.52	1798701.2400000019	10.31

W komentarzu napisz

- Jak dobrałeś progi i dlaczego - co konkretnie w danych na to wskazało?

Progi segmentów zostały dobrane na podstawie wartości monetary. Ustalono próg 300 jako granicę między klientami o niskiej i średniej wartości zakupów, natomiast próg 1500 (około 3×średnia) wyznacza grupę klientów o najwyższej wartości, stanowiących najbardziej dochodowy segment (premium). Rozkład danych jest prawoskośny, dlatego progi oparte zostały o obserwację rozkładu, a nie przez równe przedziały.

- Czy mała grupa użytkowników generuje dużą część przychodu - jaki procent użytkowników i jaki procent przychodu?

Najmniejsza grupa (premium), która stanowi tylko około 7% użytkowników, generuje prawie 25% przychodu.

- Czy widzisz coś zbliżonego do zasady Pareto?

Można zauważyć trend podobny do zasady Pareto (80/20 - około 80% efektów wynika z 20% przyczyn), ale nie jest on idealny. Około 65% użytkowników (segment standard + premium) generuje prawie 90% przychodu, co pokazuje silną koncentrację wartości w bardziej aktywnych klientach.

- Co wyniki mówią o lojalności klientów tego sklepu?

Wyniki sugerują, że sklep ma niewielką, ale bardzo wartościową grupę lojalnych klientów (premium), którzy generują znaczną część przychodów. Jednocześnie duża liczba klientów jest okazjonalna, co oznacza potencjał do zwiększenia przychodów poprzez działania zwiększające powtarzalność zakupów (np. programy lojalnościowe, personalizowane oferty).

4. Benchmark i wnioski końcowe - 4 pkt

Dlaczego to robimy

Przez całe laboratorium pisałeś zapytania analityczne. Teraz zmierzysz, jak szybko obie bazy te zapytania wykonują - i odpowiesz na pytanie będące sednem kursu: **kiedy i dlaczego ClickHouse wygrywa z PostgreSQL?** To eksperyment: hipoteza, pomiar, wynik, interpretacja.

Zasady pomiaru

Dla każdego zapytania w każdej bazie:

- uruchom zapytanie 4 razy,
- **pierwszy wynik odrzuć** - jest rozgrzewkowy,
- zapisz 3 kolejne czasy,
- oblicz średnią.

ClickHouse cache - krytyczne: przed każdą serią pomiarów uruchom SET use_query_cache = 0. Bez tego kolejne wykonania identycznego zapytania mogą zwrócić wynik z pamięci podręcznej zamiast policzyć go od nowa — czasy będą sztucznie niskie i nieporównywalne.

Trzy zapytania do zmierzenia

Użyj zapytań, które już napisałeś w zadaniach 1–3 . Nie pisz nowych, benchmark ma sens tylko wtedy, gdy mierzysz coś, co już rozumiesz.

B1 - proste - proste zapytanie agregujące z jednym GROUP BY, np.

przychód per kraj albo liczba zdarzeń per dzień – **może to być zadanie z wcześniejszych laboratoriów**

ClickHouse

```
select
    country,
    sum(price * quantity) as revenue,
    rank() over (order by sum(price * quantity) desc) as rank_no
from events
where event_type = 'purchase'
group by country
order by rank_no;
```

PostgreSQL

```
select
    country,
    sum(price * quantity) as revenue,
    rank() over (order by sum(price * quantity) desc) as rank_no
from events
where event_type = 'purchase'
group by country
order by rank_no;
```

B2 - średnie - zapytanie z lejkiem konwersji z zadania 1

```
with session_flags as (
    select
        session_id,
        countIf(event_type = 'view') > 0 as has_view,
        countIf(event_type = 'cart') > 0 as has_cart,
```

```
        countIf(event_type = 'purchase') > 0 as has_purchase
    from events
    group by session_id
)
select
    countIf(has_view) as view_sessions,
    countIf(has_cart) as cart_sessions,
    countIf(has_purchase) as purchase_sessions,
    countIf(has_cart) / nullIf(countIf(has_view), 0) as cart_per_view,
    countIf(has_purchase) / nullIf(countIf(has_cart), 0) as purchase_per_cart,
    countIf(has_purchase) / nullIf(countIf(has_view), 0) as purchase_per_view
from session_flags;
```

PostgreSQL

```
with session_flags as (
    select
        session_id,
        sum(case when event_type = 'view' then 1 else 0 end) > 0 as has_view,
        sum(case when event_type = 'cart' then 1 else 0 end) > 0 as has_cart,
        sum(case when event_type = 'purchase' then 1 else 0 end) > 0 as has_purchase
    from events
    group by session_id
)
select
    sum(case when has_view then 1 else 0 end) as view_sessions,
    sum(case when has_cart then 1 else 0 end) as cart_sessions,
    sum(case when has_purchase then 1 else 0 end) as purchase_sessions,
    sum(case when has_cart then 1 else 0 end) / nullif(sum(case when has_view then 1 else 0 end), 0) as
cart_per_view,
    sum(case when has_purchase then 1 else 0 end) / nullif(sum(case when has_cart then 1 else 0 end), 0)
as purchase_per_cart,
    sum(case when has_purchase then 1 else 0 end) / nullif(sum(case when has_view then 1 else 0 end), 0)
as purchase_per_view
from session_flags;
```

B3 - złożone - zapytanie z funkcją okna z zadania 2 lub segmentacja

RFM z zadania 3.

ClickHouse

```
with daily as (
    select
        toDate(event_time) as day,
        sum(quantity * price) as daily_revenue
    from events
    where event_type = 'purchase'
    group by day
),
daily_with_lag as (
    select
        day,
        round(daily_revenue, 2) as daily_revenue,
        round(sum(daily_revenue) over (order by day rows between unbounded preceding and current row), 2)
as cumulative_revenue,
        lagInFrame(daily_revenue, 1) over (order by day rows between unbounded preceding and current row)
as prev_day_revenue
    from daily
)
select
    day,
    daily_revenue,
```

```
    cumulative_revenue,
    prev_day_revenue,
    round(((daily_revenue / prev_day_revenue) - 1) * 100, 2) as pct_change
from daily_with_lag
order by day;
```

PostgreSQL

```
with daily as (
    select
        date(event_time) as day,
        sum(quantity * price) as daily_revenue
    from events
    where event_type = 'purchase'
    group by day
),
daily_with_lag as (
    select
        day,
        round(daily_revenue::numeric, 2) as daily_revenue,
        round(sum(daily_revenue) over (order by day)::numeric, 2) as cumulative_revenue,
        lag(daily_revenue, 1) over (order by day) as prev_day_revenue
    from daily
)
select
    day,
    daily_revenue,
    cumulative_revenue,
    prev_day_revenue,
    round((((daily_revenue / prev_day_revenue) - 1) * 100)::numeric, 2) as pct_change
from daily_with_lag
order by day;
```

Uwaga: jeśli zadania 2 lub 3 pisałeś tylko w ClickHouse, dostosuj zapytanie B3 do PostgreSQL przed pomiarem. Różnice składniowe znajdziesz w ściądze. To jest celowe zobaczysz, że logika jest taka sama, zmienia się tylko dialekt.

Tabela wyników

Wypełnij poniższą tabelę. Czasy podaj w milisekundach.

Zapytanie	Charakt.	PG p.1	PG p.2	PG p.3	PG śr.	CH p.1	CH p.2	CH p.3	CH śr.
B1 — proste	agregacja SUM z GROUP BY po kraju i funkcja okna RANK()	100	103	83	95,33	66	39	28	44,33
B2 — średnie	CTE, GROUP BY po session_id, agregacje warunkowe countIf/CASE WHEN	635	595	579	603	83	55	54	64
B3 — złożone	dwa CTE, GROUP BY po dniu, funkcje okna SUM i LAG/lagInFrame	117	85	86	96	26	30	32	29,33

W kolumnie „Charakterystyka" wpisz jednym zdaniem, co zapytanie robi: ile kolumn czyta czy używa GROUP BY, COUNT(DISTINCT), funkcji okna, CTE.

Analiza planu wykonania

Dla zapytania **B1** i **B3** uruchom:

```
-- PostgreSQL — wykonuje zapytanie i pokazuje realne czasy
EXPLAIN ANALYZE <zapytanie>;
```

```
-- ClickHouse — pokazuje plan bez wykonania
EXPLAIN <zapytanie>;
```

Nie jest wymagana pełna analiza techniczna. Napisz po **2–3 zdania** dla każdego z czterech planów (B1 w PG, B1 w CH, B3 w PG, B3 w CH):

- gdzie w planie widać agregację i sortowanie,
- czy plan B3 jest wyraźnie bardziej rozbudowany niż B1,
- czym plan ClickHouse różni się od planu PostgreSQL - na co zwróciłeś uwagę.

B1 - PostgreSQL

```
QUERY PLAN
1  Sort  (cost=16938.33..16938.36 rows=10 width=19) (actual time=54.333..56.008 rows=10 loops=1)
2    Sort Key: (rank() OVER (?))
3    Sort Method: quicksort  Memory: 25kB
4    ->  WindowAgg  (cost=16937.99..16938.17 rows=10 width=19) (actual time=54.320..56.001 rows=10 loops=1)
5          ->  Sort  (cost=16937.99..16938.02 rows=10 width=11) (actual time=54.311..55.986 rows=10 loops=1)
6                Sort Key: (sum((price * (quantity)::double precision))) DESC
7                Sort Method: quicksort  Memory: 25kB
8                ->  Finalize GroupAggregate  (cost=16935.29..16937.83 rows=10 width=11) (actual time=54.298..55.979 rows=10 loops=1)
9                      Group Key: country
10                     ->  Gather Merge  (cost=16935.29..16937.63 rows=20 width=11) (actual time=54.290..55.969 rows=30 loops=1)
11                           Workers Planned: 2
12                           Workers Launched: 2
13                           ->  Sort  (cost=15935.27..15935.29 rows=10 width=11) (actual time=46.794..46.800 rows=10 loops=3)
14                                 Sort Key: country
15                                 Sort Method: quicksort  Memory: 25kB
16                                 Worker 0:  Sort Method: quicksort  Memory: 25kB
17                                 Worker 1:  Sort Method: quicksort  Memory: 25kB
18                                 ->  Partial HashAggregate  (cost=15935.00..15935.10 rows=10 width=11) (actual time=46.752..46.759 rows=10 loops=3)
19                                       Group Key: country
20                                       Batches: 1  Memory Usage: 24kB
21                                       Worker 0:  Batches: 1  Memory Usage: 24kB
22                                       Worker 1:  Batches: 1  Memory Usage: 24kB
23                                       ->  Parallel Seq Scan on events  (cost=0.00..15728.33 rows=20667 width=15) (actual time=0.066..43.094 rows=16778 loops=3)
24                                             Filter: (event_type = 'purchase'::text)
25                                             Rows Removed by Filter: 316556
26 Planning Time: 0.392 ms
27 Execution Time: 56.133 ms
```

Agregacja jest w Partial HashAggregate i Finalize GroupAggregate (wiersze 18, 8), sortowanie w Sort (wiersze 5, 13). PostgreSQL przeskanował całą tabelę wierszowo (Parallel Seq Scan, wiersz 23), dopiero po odczycie odfiltrował rekordy z event_type = 'purchase'. Żeby przyspieszyć skan, podzielił pracę między dwóch workerów, których wyniki scalił w Gather Merge (wiersz 10).

B1 - ClickHouse

```
explain
1  Expression (Project names)
2    Sorting (Sorting for ORDER BY)
3      Expression ((Before ORDER BY + Projection))
4        Window (Window step for window 'ORDER BY sum(multiply(__table1.price, __table1.quantity)) DESC')
5          Sorting (Sorting for window 'ORDER BY sum(multiply(__table1.price, __table1.quantity)) DESC')
6            Expression (Before window functions)
7              Aggregating
8                Expression (Before GROUP BY)
9                  Expression ((WHERE + Change column names to column identifiers))
10                   ReadFromMergeTree (ds_lab.events)
```

Agregacja jest w Aggregating (wiersz 7), sortowanie w Sorting (wiersze 2, 5). ClickHouse czyta tylko potrzebne kolumny bezpośrednio z ReadFromMergeTree (wiersz 10), filtrowanie WHERE odbywa się już podczas odczytu (wiersz 9), nie po nim. Plan jest krótszy i płaski w

porównaniu do postgresa.

B3 - PostgreSQL

```
QUERY PLAN
1  Subquery Scan on daily_with_lag  (cost=18261.34..26478.81 rows=49512 width=108) (actual time=43.681..47.786 rows=180 loops=1)
2    ->  WindowAgg  (cost=18261.34..25736.13 rows=49512 width=76) (actual time=43.676..47.619 rows=180 loops=1)
3          ->  Finalize GroupAggregate  (cost=18261.34..24374.55 rows=49512 width=12) (actual time=43.555..47.190 rows=180 loops=1)
4                Group Key: (date(events.event_time))
5                ->  Gather Merge  (cost=18261.34..23548.98 rows=41334 width=12) (actual time=43.515..47.108 rows=540 loops=1)
6                      Workers Planned: 2
7                      Workers Launched: 2
8                      ->  Partial GroupAggregate  (cost=17261.31..17777.99 rows=20667 width=12) (actual time=37.182..40.184 rows=180 loops=3)
9                            Group Key: (date(events.event_time))
10                           ->  Sort  (cost=17261.31..17312.98 rows=20667 width=16) (actual time=37.133..38.759 rows=16778 loops=3)
11                                 Sort Key: (date(events.event_time))
12                                 Sort Method: quicksort  Memory: 1507kB
13                                 Worker 0:  Sort Method: quicksort  Memory: 1014kB
14                                 Worker 1:  Sort Method: quicksort  Memory: 982kB
15                                 ->  Parallel Seq Scan on events  (cost=0.00..15780.00 rows=20667 width=16) (actual time=0.056..33.563 rows=16778 loops=3)
16                                       Filter: (event_type = 'purchase'::text)
17                                       Rows Removed by Filter: 316556
18 Planning Time: 0.247 ms
19 Execution Time: 47.938 ms
```

Agregacja jest w Partial GroupAggregate i Finalize GroupAggregate (wiersze 8, 3), funkcja okna w WindowAgg (wiersz 2), sortowanie w Sort (wiersz 10). Plan jest wyraźnie bardziej rozbudowany niż B1, pojawia się tam Subquery Scan on daily_with_lag odpowiadający CTE, a zużycie pamięci na sortowanie wzrosło do 1507 kB wobec 25 kB w B1. Podobnie jak w B1, postgres skanuje całą tabelę wierszowo i filtruje po odczycie.

B3 - ClickHouse

```
explain
1  Expression ((Project names + (Before ORDER BY + (Projection + (Change column names to column identifiers + (Project names + Projection)))) [lifted up part]))
2    Sorting (Sorting for ORDER BY)
3      Expression ((Before ORDER BY + (Projection + (Change column names to column identifiers + (Project names + Projection)))))
4        Window (Window step for window 'ORDER BY __table2.day ASC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW')
5          Expression ((Before WINDOW + (Change column names to column identifiers + (Project names + Projection))) [lifted up part])
6            Sorting (Sorting for window 'ORDER BY __table2.day ASC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW')
7              Expression ((Before WINDOW + (Change column names to column identifiers + (Project names + Projection))))
8                Aggregating
9                  Expression (Before GROUP BY)
10                   Expression ((WHERE + Change column names to column identifiers))
11                     ReadFromMergeTree (ds_lab.events)
```

Agregacja jest w Aggregating (wiersz 8), funkcja okna w Window (wiersz 4), sortowanie w Sorting (wiersze 2, 6). Plan jest dłuższy niż B1 o kilka kroków, ale struktura pozostaje podobna. ClickHouse nadal czyta tylko potrzebne kolumny z ReadFromMergeTree i filtruje podczas odczytu.

Refleksja końcowa – obowiązkowa

Na podstawie pomiarów i planów napisz **5–8 zdań** odpowiadając na poniższe pytania. To jest najważniejszy element całego laboratorium.

- Czy różnica czasu między bazami rosła wraz ze złożonością zapytania?

Różnica czasu między bazami rosła wraz ze złożonością, ale nie liniowo. Najwieksza przewaga ClickHouse ujawniła się przy B2, gdzie postgres potrzebował średnio 603 ms, a ClickHouse tylko 64 ms, czyli prawie 10x szybciej. Przy B1 i B3 przewaga była mniejsza, odpowiednio ok 2x i 3x. Zaskakujące jest że B3 z dwoma CTE i funkcjami okna okazało się szybsze niż B1 zarówno w PostgreSQL jak i ClickHouse, może byc to spowodowane tym, że B3 agreguje dane do poziomu dziennego przed zastosowaniem funkcji okna, co redukuje liczbę przetwarzanych wierszy.

- Przy którym zapytaniu przewaga ClickHouse była największa i jak to tłumaczysz?

Największa przewaga ClickHouse przy zapytaniu B2 wynika z tego, że lejek wymaga przeskanowania całej tabeli i agregacji po session_id, co w postgresie oznacza pełny Seq Scan przez wszystkie kolumny. ClickHouse czyta tylko te kolumny których potrzebuje (session_id i

event_type) i filtruje dane już podczas odczytu z dysku, zanim trafią do pamięci.

- Co plany wykonania mówią o tym, dlaczego ClickHouse jest szybszy dla tego rodzaju zapytań?

Plany wykonania potwierdzają tę różnicę. PostgreSQL zawsze zaczyna od Parallel Seq Scan, czyli czyta wszystkie kolumny wiersz po wierszu i filtruje dopiero po odczycie. Clickhouse zaczyna od ReadFromMergeTree i filtruje podczas odczytu, operując na danych kolumnowych co przy zapytaniach agregujących kilka kolumn z milionów wierszy daje ogromną przewagę.

- Gdybyś był architektem systemu danych w firmie e-commerce obsługującej miliony zdarzeń dziennie - dla jakich zadań wybrałbyś ClickHouse, a dla jakich PostgreSQL? Uzasadnij konkretnie.

Jako architekt systemu e-commerce wybrałabym ClickHouse do wszystkich zapytań analitycznych czyli raportowanie przychodow, lejki konwersji, segmentacja użytkowników, szczególnie gdy dane liczone są w milionach wierszy dziennie. PostgreSQL zostawiłabym do operacji transakcyjnych gdzie liczy się spójność danych czyli np. rejestrowanie zamówień, aktualizacja stanów magazynowych, obsluga płatności, bo tam model wierszowy i gwarancje ACID są ważniejsze niż szybkość odczytu.

Refleksja jest oceniana jako osobny punkt (1 pkt). Oczekujemy własnego rozumowania opartego na zmierzonych danych - nie powtórzenia treści instrukcji.

Zadanie dodatkowe — dla chętnych (bez dodatkowych punktów)

Jeżeli skończyłeś wcześniej, wybierz jedną z poniższych opcji.

Opcja A — agregacja tygodniowa - przepisz zapytanie z zadania 2 tak, aby grupowało dane tygodniowo. W ClickHouse użyj toStartOfWeek, w PostgreSQL DATE_TRUNC('week', ...). Czy wyniki są identyczne? Czy jest różnica w składni?

Opcja B — uniq vs uniqExact - w ClickHouse policz liczbę unikalnych użytkowników per dzień używając najpierw uniqExact, potem uniq. Zmierz czas obu wersji. O ile uniq jest szybsze i jak duży jest błąd przybliżenia? Kiedy w prawdziwym projekcie zaakceptowałbyś wynik przybliżony?

Opcja C — własne pytanie analityczne - zadaj sobie pytanie dotyczące tabeli events, którego jeszcze nie badałeś, i odpowiedz na nie zapytaniem SQL. Napisz, skąd wziął się pomysł, jak podszedłeś do problemu i co odkryłeś.

Co jest oceniane

Element	Punkty
Zadanie 1 — lejek konwersji w ClickHouse + komentarz z wyjaśnieniem countIf	2
Zadanie 2 — funkcje okna: ranking i trend z LAG	2
Zadanie 3 — segmentacja RFM z własnym doborem progów	2
Zadanie 4 — pomiary benchmarkowe z wypełnioną tabelą	1
Zadanie 4 — analiza planów EXPLAIN	1
Zadanie 4 — refleksja końcowa	1
Jakość komentarzy i interpretacji we wszystkich zadaniach	1
Razem	10

Napisanie działającego kodu to warunek konieczny, nie wystarczający. Jeżeli uruchomiłeś zapytanie i dostałeś wynik, ale nie rozumiesz, co z niego wynika dla biznesu, oddałeś połowę zadania.